

Flexible Notification for Collaborative Systems

Haifeng Shen and Chengzheng Sun

School of Computing and Information Technology

Griffith University

Brisbane, Qld 4111, Australia

{Hf.Shen, C.Sun}@cit.gu.edu.au

ABSTRACT

Notification is an essential feature in collaborative systems, which determines a system's capability and flexibility in supporting different kinds of collaborative work. In the past years, various notification strategies have been designed for different systems. However, the design of notification components has been ad hoc, and the techniques used for supporting notification have been application-dependent. In this paper, we contribute a flexible notification framework that can be used to describe and compare a range of notification strategies used in existing collaborative systems, and to guide the design of notification components for new collaborative systems. The framework has been applied to the design of a notification component for a group editor, which uses a single notification mechanism to support various notification policies for meeting both real-time and non-real-time collaboration needs. In addition, a new operational transformation control algorithm has been devised in combination with the notification component, which is significantly simpler and more efficient than existing algorithms.

Keywords: Collaborative system, notification, group editor, concurrency control, operational transformation.

INTRODUCTION

Notification is an essential feature in collaborative systems, which determines when, what, and how updates made by one user are propagated, applied, and reflected on other users' interfaces. It is this feature that distinguishes collaborative systems from traditional multi-user systems, such as database management systems and timesharing operating systems, where a user is normally not notified of the actions performed by other users [6].

Notification plays an important role in determining a system's capability and flexibility in supporting different kinds of collaborative work. If a system has adopted a notification strategy that *frequently* propagates one user's actions to others, then this system is capable of supporting *real-time* (or *synchronous*) collaborative work, where multiple users can

collaborate at *the same time*. In contrast, if a system has adopted a notification strategy that *infrequently* propagates one user's actions to others, then this system is more suitable for supporting *non-real-time* (or *asynchronous*) collaborative work, where multiple users can collaborate at *different times*. Usually, one collaborative system uses only one notification strategy, and existing collaborative systems have been classified to be either real-time or non-real-time. However, there is no technical reason that a system cannot use multiple notification strategies to support both real-time and non-real-time collaborative work.

In the past years, various notification strategies have been designed for different collaborative systems to meet their special collaboration needs. For example, the *REDUCE* (Real-time Distributed Unconstrained Cooperative Editing) editor [16] has focused on achieving high responsiveness but adopted a simple notification strategy: operations performed on the shared document by one user is immediately and automatically propagated to other users and reflected on other users' interfaces as soon as possible. *REDUCE* also provides some limited support for controlling the granularity of propagated operations: operations can be at the level of either individual characters or arbitrary length strings [8]. In the Internet-based real-time chatting tool *ICQ* [7], a message is sent as soon as the user hits the *Send* button, and reflected on other users' interfaces on arrival. In the version control system *CVS* (Concurrent Versions System) [2], a user can edit the copied document as long as s/he wishes. Updates made on the shared document by one user is not made available to others until the user manually issues the *commit* command [2], and updates made by other users are not integrated into the local copy unless the user manually issues the *update* command [2]. In all these collaborative systems, the design of notification components has been ad hoc, and techniques used for supporting notification have been application-dependent. Little research has been done on generic aspects of notification to provide an integrated and unified view of existing notification strategies, and to provide a guideline for the design of new notification strategies.

In collaborative applications, notification can be used to support exchanging messages. For example, messaging systems like *Email* or *ICQ* [7] use notification to support message exchange. In addition, notification can be used to sup-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

CSCW'02, November 16-20, 2002, New Orleans, Louisiana, USA.

Copyright 2002 ACM 1-58113-560-2/02/0011 ...\$5.00.

port group awareness. For example, *ICQ* [7] uses notification to inform others of a participant's status change (e.g., online/offline), and *NESSIE* [12] uses notification to inform others of a task that has been completed by a participant. Notification can also be used to support concurrency control in order to maintain consistency of shared artifacts. For example, *REDUCE* [16] uses propagation of operations [8] and operational transformation [16, 15] as the concurrency control mechanism to maintain consistency among replicated copies, and *CVS* [2] uses propagation of documents, document merging [14], and mutual exclusive locking as the concurrency control mechanism to maintain consistency among replicated copies.

In this paper, we contribute a flexible notification framework that can be used to describe and compare a range of notification strategies used in existing collaborative systems, and to guide the design of notification components for new collaborative systems. In the proposed framework, the notification policy that determines when and what to notify is separated from the notification mechanism that determines how to notify. The notification policy part of the framework consists of a set of basic and generic parameters which can be used to define a spectrum of notification policies suitable for meeting different collaboration needs. The notification mechanism part consists of a set of basic and generic technical components which can be used to support various notification policies. Finally, we show how to apply this framework to the design of a flexible notification component for a group editor, which can support various notification policies for meeting a range of collaboration needs. In addition, a new operational transformation control algorithm has been devised in combination of the notification component for concurrency control, which is significantly simpler and more efficient than existing algorithms.

The rest of this paper is organized as follows. The following section presents a flexible notification framework. Then the next two sections describe the design of a notification component for a group editor by means of the proposed framework. Comparison to related work is discussed in the following section. Finally the paper is concluded with a summary of our major contributions and future work.

A FLEXIBLE NOTIFICATION FRAMEWORK

The proposed framework consists of a notification policy part and a notification mechanism part. We first discuss the policy part.

To achieve flexible notification, we differentiate two directions of notifications. One is the Outgoing Notification direction (*ON*), which is about when, what, and how updates made by a user are propagated to others. The other is the Incoming Notification direction (*IN*), which is about when, what, and how updates made by other users are accepted and reflected on a local user's interface. Two parameters *frequency* and *granularity* are provided to define various notification policies.

Notification frequency

The frequency parameter determines the “when” aspect of notification, that is, when a notification is to be propagated/accepted.

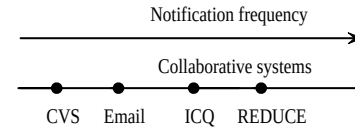


Figure 1: Relationship between notification frequency and collaborative systems

According to the frequency of notification, we can sort collaborative systems in a linear order, with less frequently notifying systems ordered before more frequently notifying systems. As shown in Figure 1, non-real-time collaborative systems, such as *CVS* and the *Email* system, occupy the left spectrum of notification frequencies, and real-time collaborative systems, such as *ICQ* and *REDUCE*, are using the right spectrum of notification frequencies.

System-triggered	I: Instant
	S: Scheduled
User-triggered	U: User-controlled

Figure 2: Notification frequency parameter values

The frequency parameter may take different values, which are shown in Figure 2. Values for *ON/IN* directions, denoted as *ONF/INF*, can be classified into two categories: system-triggered and user-triggered. In system-triggered notification, the system automatically propagates/accepts notification. If $ONF = I$, every update made by a user would be instantly propagated, and if $INF = I$, every update made by another user would be instantly accepted and reflected on a local user interface. *ICQ* is an example of using $INF = I$, where arrived messages are instantly reflected on the user interface. If $ONF/INF = S$, the system propagates/accepts notifications as scheduled. Scheduling is application-dependent and may be based on some rules that are related to external events. Scheduling rules could be either implicitly built in the application or explicitly configured by users. For example, in *REDUCE*, the *INF* has the following system built-in scheduling rule: a remote operation is accepted only when it is causally ready. In many email programs, new messages will be checked out in the user-specified time interval. In user-triggered notification, notification is propagated/accepted by the user's explicit commands. For example, in *CVS*, updates made by a user are not made available to others unless the user issues the *commit* command [2].

Different collaborative systems usually take different notification frequency parameter values. For example, as shown

in Figure 3, in *CVS*, all participants have the same parameter values: $ONF/INF = U$. So updates made by a user are not propagated unless s/he issues the *commit* command, and these updates are not applied to another user's copy unless that user issues the *update* command. In *ICQ*, all participants have the same parameter values: $ONF = U$ and $INF = I$. So a message is not sent out unless the user hits the *Send* button, and when the message arrives at a remote site, it will be displayed on the remote user's interface immediately. In *REDUCE*, all participants have the same parameter values: $ONF = I$ and $INF = S$. So each operation generated at a site is instantly propagated and when it arrives at another site, it is executed when causally ready.

	CVS	Email	ICQ	REDUCE
<i>ONF</i>	<i>U</i>	<i>U/S</i>	<i>U</i>	<i>I</i>
<i>INF</i>	<i>U</i>	<i>S/S</i>	<i>I</i>	<i>S</i>

Figure 3: Frequency parameter values used in example systems

A collaborative system may allow different participants to have different notification parameter values. For example, in the *Email* system, consider two users *A* and *B* work this way. Both of them have specified a time interval to check emails. Suppose *B* sends *A* an email but *A* was away from her/his office. Before her/his departure, *A* has configured her/his email system with the capability of automatically replying every received email with a pre-described message telling s/he is on vacation. When *B*'s email is checked out in *A*'s email system, it will be automatically replied with the pre-described message. So *A*'s frequency parameter values are: $ONF = S$; $INF = S$ while *B*'s frequency parameter values are: $ONF = U$; $INF = S$.

Notification granularity

The granularity parameter determines the “what” aspect of notification, that is, which updates is going to be propagated/accepted.

We use ONG/ING to denote the granularity parameter for ON/IN directions respectively. The granularity parameter may take two values *A* (All) and *S* (Selective). If $ONG/ING = A$, all updates accumulated since last propagation/acceptance are propagated/accepted. Most existing collaborative systems adopt this granularity policy. For example, in *CVS*, when a user issues the *commit* command, all updates accumulated since the last *commit* operation are propagated. If $ONG/ING = S$, users can select updates among those that have been accumulated since the last propagation/acceptance to propagate/accept.

Selective notification is flexible and usually triggered by users with explicit selection criteria. But if selection criteria can be pre-defined, notification could also be automatically triggered by the system. Selection criteria are application dependent. Some typical examples are:

- Time criterion: a user can selectively propagate/accept updates made within a certain period of time.

- Object criterion: a user can selectively propagate/accept updates made to certain objects.
- Type criterion: a user can selectively propagate/accept certain types of updates.
- Version criterion: a user can selectively propagate/accept updates made in certain versions of a shared artifact.
- User criterion: a user can selectively notify certain users of her/his updates or selectively accept updates made by certain users.

These criteria can be used as filters for the notifying site to select certain updates to notify, or for the notified site to select certain notifications to accept. Carzaniga et al. has presented a systematical approach in [4] to select appropriate collection of events using filters and patterns. Although its primary goal is different from selective notification, most of the ideas can be directly applied or extended to achieve selective notification.

An example of $ONG = S$ is the *Netmeeting* [9] conferencing system where a user's desktop environment can be shared with others. In this system, a user is allowed to notify others of all updates made in her/his desktop environment, or selectively notify others of updates made in particular applications. So the selection is based on the *Object* criterion. An example of $ING = S$ is *CVS* where a user is able to specify a version number to select updates made till that version to be applied into his copy. So the selection is based on the *Version* criterion.

In general, if the frequency of notification is high, each notification tends to have a small granularity, and $ONG/ING = A$ is usually used. If the frequency of notification is low, each notification tends to have a large granularity if $ONG/ING = A$, or have a small granularity if $ONG/ING = S$. The notification frequency and granularity parameters can be used to define a range of notification policies used in existing collaborative systems, and to guide the designs of notification policies for new systems.

Notification mechanism

In the policy part, we have explored the “when” and “what” aspects of notification. This section focuses on the “how” aspect, which is about the notification mechanism part.

To support a range of notification policies, a notification buffer is needed to buffer updates in order to allow users to select any updates to propagate/accept at any time. Furthermore, in order to support separated notification policies for *ON* and *IN*, two separate buffers Outgoing Buffer (*OB*) and Incoming Buffer (*IB*) should be used as notification buffers. *OB* stores all locally performed updates so that a user is able to select any local updates to propagate at any time. *IB* stores all remote propagated updates so that a user is able to select any remote updates to accept.

An Outgoing Notification Executor(*ONE*) component and an Incoming Notification Executor(*INE*) component are needed to carry out various outgoing and incoming notification policies respectively. In particular, the *ONE* component manages

the *OB* buffer, selects a collection of updates from *OB*, and properly reformulates the selected updates to propagate. The *INE* component manages the *IB* buffer, selects a collection of updates from *IB*, and properly reformulates the selected updates to accept.

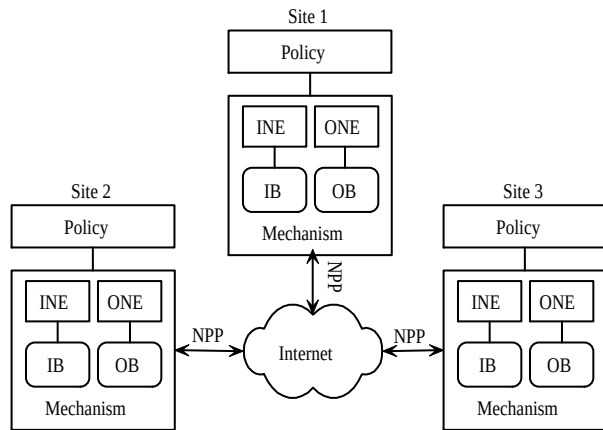


Figure 4: A flexible notification framework

A very important component in notification mechanism is Notification Propagation Protocol (*NPP*), which is needed for propagating updates from the *OB* at the notifying site to the *IB* at the notified site. There are two alternatives to design the protocol. One is the *poll* approach in which the notified site polls updates from the *OB* at the notifying site. An example of using this approach is *CVS* in which a user uses the *update* command to poll updates into her/his working copy. The other is the *Push* approach in which the notifying site pushes updates into the *IB* at the notified site. The *Publish/Subscribe* paradigm [3, 1] is a typical example of using this approach, where the publisher pushes certain messages to certain subscribers who are interested in.

In summary, the notification framework is shown in Figure 4. The policy part makes notification policies. The notification mechanism part consists of notification buffers and notification executors for executing notification policies, and the notification propagation protocol for supporting communication and interaction between the notifying site and notified sites. In the following sections, we demonstrate how to apply the proposed notification framework to the design of a notification component for a group editor, which can support various notification policies for meeting a range of collaboration needs.

A NOTIFICATION COMPONENT FOR A GROUP EDITOR

A group editor is a typical collaborative system where multiple participants can edit a shared document that is replicated among participating sites. In existing group editors, notification policies for meeting real-time collaboration needs are used in real-time systems like *REDUCE* while notification policies for meeting non-real-time collaboration needs are used in non-real-time systems like *CVS*.

Notification policies

Our objective is to design a notification component for a group editor, which uses a single generic notification mechanism to support a range of notification policies for meeting both real-time and non-real-time collaboration needs. Such an integrated system is desirable in practice because real-time and non-real-time systems could be both needed in developing a project at different stages or under different circumstances. A real-time system is required when a group of users need frequent interaction to achieve some common goal (e.g., collaboratively debugging a computer program). A non-real-time system is required if real-time collaboration is not achievable, or collaborators do not want to coordinate with one another interactively (e.g., collaborators concurrently write different chapters for a book). There are two possibilities that real-time collaboration is not achievable. One is network connection is discontinuous/unreliable (e.g., in mobile and wireless computing environments). Another is collaborators are located in different time zones. However, it is neither easy for users to be familiar with multiple systems nor convenient to manually switch among different systems from time to time. It is highly desirable for people to use the same editor to work in non-real-time collaboration and real-time collaboration modes.

Real-time systems like *REDUCE* capture updates to the shared document as operations (or events). When a user wants to notify others of an update, the corresponding operation will be propagated to other sites [8]. Propagation of operations in real-time systems is very frequent. The timestamping technique is usually used to capture concurrent relationships among operations, and the operational transformation technique [16, 15] is usually used for concurrency control to maintain consistency among replicas. Each propagation message in real-time systems may contain a small collection of operations, normally one operation. For each received operation, a sophisticated transformation control algorithm like *GOTO* [15] needs to be executed to get the correct execution form of the operation to achieve consistency. While this approach works well for real-time systems, it is unsuitable for non-real-time systems. First of all, non-real-time systems tend to have a large and nondeterministic number of participants, which makes it unrealistic to timestamp operations with a fixed size of state vectors. Moreover, in non-real-time systems, propagation messages are less frequent but each message normally contains a large number of operations. It would be wasteful and inefficient to timestamp individual operations and trigger a sophisticated transformation control algorithm for each received operation.

Non-real-time systems like *CVS*, do not capture updates in the form of operations/events. Instead, each *ON* notification transfers the updated document into the repository and executes a time-consuming text differentiation algorithm like *diff* [10] there to generate *deltas* [2] between the updated document and its original version. Each *IN* notification transfers the original version and versions made by remote users

from the repository, and executes a time-consuming merging algorithm like *diff3* [10] at the notified site to merge updates made at remote sites into the local copy. This approach cannot be used for real-time systems because it is unacceptable to transfer entire documents over the network back and forth, and execute time-consuming [10] text differentiation and merging algorithms just for notifying a single update.

To support a range of notification policies by a single group editor, we devise a flexible notification component with a generic notification mechanism to support various notification policies. Because concurrency control is a key issue in group editors, we also show how concurrency control is integrated in the proposed notification mechanism.

Notification algorithms

OB stores operations sequentially generated and executed at the local site. So operations in an *OB* are sorted in the causal order from left to right. *IB* stores operations propagated from remote sites. These operations have been contextually serialized in the sense that for any O_x and O_y in an *IB*, if O_x is at the left side of O_y , then it must be: (1) O_x is contextually preceding O_y [16], or (2) O_x is concurrent with O_y , but O_y has taken into account O_x 's effect by operational transformation [16]. How to achieve contextual serialization will be explained in the next section. Operations in the *OB* and *IB* at the same site are concurrent [16]: given any O_x and O_y , where O_x is in *IB* and O_y is in *OB*, O_x and O_y must be concurrent operations.

To propagate any operation O_x in OB_l at site l , the following two steps should be done. First, O_x must be transposed to the place between the last propagated operation and the first unpropagated operation in OB_l in order to support various outgoing notification granularities. O_x is transformed into the form that O_x is contextually preceding all unpropagated operations so that when O_x arrives at a remote site r , there is no operation in IB_r , which has not arrived but casually before O_x in OB_l . This step is done by the *LTranspose* procedure, which is based on operational transformation functions.

There are two types of primitive transformation functions [16]: one is the *Inclusion Transformation* function – $IT(O_a, O_b)$, which transforms operation O_a against operation O_b in such a way that the impact of O_b is effectively included in the parameters of the output operation O'_a ; and the other is the *Exclusion Transformation* function – $ET(O_a, O_b)$, which transforms O_a against O_b in such a way that the impact of O_b is effectively excluded from the parameters of the output operation O'_a . *LTranspose*(L) procedure is defined to transpose (i.e., transform and shift) the right-most operation in L to the left-most position in L .

Procedure 1 *LTranspose*(L)

```
{ for ( $k = |L|$ ;  $k > 1$ ;  $k--$ )
  { /* transform */
     $L[k] := ET(L[k], L[k-1]);$ 
     $L[k-1] := IT(L[k-1], L[k]);$ 
  } /* shift */
```

```
 $O := L[k];$ 
 $L[k] := L[k-1];$ 
 $L[k-1] := O;$ 
}
```

In the second step, O_x must be transformed against all operations in IB_l before propagation. The propagation protocol ensures operations in IB_r must be either operations in OB_l that have been propagated before O_x , or operations that have also appeared in IB_l . Consequently, when O_x arrives at a remote site r , it can be simply appended in IB_r . On the other hand, operations in IB_l must also be transformed against the concurrent operation O_x in order to preserve their intentions [16]. Therefore operations in OB_l and IB_l need to be transformed against each other symmetrically. The *SLOT* (Symmetric Linear Operation Transform) control algorithm is defined as follows.

Procedure 2 *SLOT*(L_1, L_2)

```
{ for ( $i = 1$ ;  $i \leq |L_2|$ ;  $i++$ )
  for ( $j = 1$ ;  $j \leq |L_1|$ ;  $j++$ )
     $SIT(L_2[i], L_1[j]);$ 
}
```

The *SIT*(O_a, O_b) (Symmetric Inclusive Transformation) procedure is defined to inclusively transform O_a and O_b symmetrically.

Procedure 3 *SIT*(O_a, O_b)

```
{  $O'_a := IT(O_a, O_b);$ 
   $O'_b := IT(O_b, O_a);$ 
   $O_a := O'_a;$ 
   $O_b := O'_b;$ 
}
```

To execute any operation O_y in IB_l at site l , the following two steps need to be done. First, O_y must be transposed to the left-most side of IB_l in order to support various incoming notification granularities. O_x is transformed into the form that O_x is contextually preceding the rest of operations in IB_l so that it will be executed in the right context. Second, O_y is transformed with unpropagated operations in OB_l to preserve their intentions. Operations in OB_l should be concurrent with operations in IB_l . But O_y must have been transformed with those propagated operations in OB_l before their propagations. So O_y only needs to be transformed with those unpropagated operations in OB_l .

In summary, the *AnyONE* algorithm for the *ONE* component is defined as follows to propagate a list of selected operations $[O_{N_1}, \dots, O_{N_k}]$ from *OB*.

Algorithm 1 *AnyONE*($[O_{N_1}, \dots, O_{N_k}]$)

Suppose $OB = [O_1, \dots, O_m, \dots, O_n]$. A pointer called *ILPO* (Identifier of Last Propagated Operation) is maintained to remember the last propagated operation in *OB*. $ILPO = ID(O_m)$ where $ID(O_m)$ is O_m 's *ID*. So O_i ($1 \leq i \leq m$) are called *propagated* operations, and O_j ($m < i \leq n$) are called *unpropagated* operations. The following steps are executed to propagate the list of operations $[O_{N_1}, \dots, O_{N_k}]$ where $m < N_1, \dots, N_k \leq n$.

1. Transpose $O_{N_1}, \dots, O_{N_k}$ to the place between O_m and O_{m+1} by repeatedly using the *LTranspose* procedure:
 $\text{for } (i = 1; i \leq k; i++) \{ LTranspose([O_{m+1}, \dots, O_{N_i}]); \}$
After then, OB becomes $[O_1, \dots, O_m, O'_{N_1}, \dots, O'_{N_k}, O'_{U_1}, \dots, O'_{U_l}]$ where $m < U_1, \dots, U_l \leq n$ and $U_1, \dots, U_l \neq N_1, \dots, N_k$.
2. If IB is empty, skip this step. Otherwise suppose $IB = [O_1, \dots, O_r]$. $SLOT([O'_{N_1}, \dots, O'_{N_k}], IB)$ needs to be executed. After then, $[O'_{N_1}, \dots, O'_{N_k}]$ becomes $[O''_{N_1}, \dots, O''_{N_k}]$, OB becomes $[O_1, \dots, O_m, O'_{N_1}, \dots, O'_{N_k}, O'_{U_1}, \dots, O'_{U_l}]$, and IB becomes $[O'_1, \dots, O'_r]$.
3. Propagate the list of operations $[O''_{N_1}, \dots, O''_{N_k}]$ in a notification message. The pointer $ILPO$ is set to $ID(O''_{N_k})$.

Correspondingly, the *AnyINE* algorithm for the *INE* component is defined as follows to execute a list of selected operations $[O_{E_1}, \dots, O_{E_k}]$ where $1 \leq E_1, \dots, E_k \leq n$ from $IB = [O_1, \dots, O_n]$.

Algorithm 2 *AnyINE* $([O_{E_1}, \dots, O_{E_k}])$

1. Transpose $O_{E_1}, \dots, O_{E_s}$ to the left-most position in IB by repeatedly using the *LTranspose* procedure:
 $\text{for } (i = 1; i \leq k; i++) \{ LTranspose([O_1, \dots, O_{E_i}]); \}$
After then, IB becomes $[O'_{E_1}, \dots, O'_{E_k}, O'_{U_1}, \dots, O'_{U_l}]$ where $1 < U_1, \dots, U_l \leq n$ and $U_1, \dots, U_l \neq E_1, \dots, E_k$.
2. If OB contains no unpropagated operation, skip this step. Otherwise suppose $OB = [O_1, \dots, O_m, \dots, O_n]$ with $ILPO = ID(O_m)$, $SLOT([O'_{E_1}, \dots, O'_{E_k}], [O'_{E_1}, \dots, O'_{E_k}])$ needs to be executed. After then, $[O'_{E_1}, \dots, O'_{E_k}]$ becomes $[O''_{E_1}, \dots, O''_{E_k}]$, IB becomes $[O''_{E_1}, \dots, O''_{E_k}, O'_{U_1}, \dots, O'_{U_l}]$, OB becomes $[O_1, \dots, O_m, O'_{m+1}, \dots, O'_n]$.
3. Execute operations $O''_{E_1}, \dots, O''_{E_k}$ sequentially, and remove them from IB . After then, $IB = [O'_{U_1}, \dots, O'_{U_l}]$. A pointer called *ILAO* (Identifier of Last Accepted Operation) is maintained to remember the last accepted operation from IB . So *ILAO* is set to $ID(O''_{E_k})$.

The *ILPO* and *ILAO* pointers are needed in notification propagation protocol, which is explained as follows.

Notification propagation protocol

We adopt the *push* approach for the notification propagation protocol in order to efficiently support a wide range of notification policies because the *poll* approach is not suitable for supporting those notification policies for meeting real-time collaboration needs [5].

Notification is contextually serialized. The serialization can be implemented in many ways. We use *Notifier*-based solution as an example to illustrate our solution. The *Notifier* is a notification server [11], which acts as a centralized serialization point and a message relaying agent.

Sequential propagation An option is sequential propagation, where only one notification is outstanding at any time. In this case, the *Notifier* manages a token that circulates among

participating sites, and a waiting queue to queue token requests. Only the site that has been granted the token is able to propagate a notification.

Before propagating a notification, the notifying site sends a *Token-Request* message to the *Notifier*, waiting for the *Token-Grant* message from the *Notifier*. After being granted the token, the site propagates the notification piggybacked with the *Token-Release* message to the *Notifier*. When the *Notifier* receives the notification and the *Token-Release* message, it forwards the notification to all interested notified sites. By using the *Notifier* as a message relaying agent, causal relationships among notifications are automatically guaranteed. The *Notifier* also dispatches the token according to the waiting queue, and sends the *Token-Grant* message to the next granted site. When receiving a notification, the notified site simply appends operations piggybacked in the notification into its IB .

Sequential propagation simplifies concurrency control because for any O_x in OB_k at site k , all operations that are concurrent with O_x must only be in IB_k . As a result, for concurrency control, O_x only needs to be transformed against all operations in IB_k before propagation. On the other hand, for any O_y in IB_k , all operations that are concurrent with O_y must only be in OB_k . As a result, for concurrency control, O_y only needs to be transformed against all operations in IB_k before execution.

However, sequential propagation is inefficient for supporting notification policies for meeting real-time collaboration needs. For propagating each notification that may contain only one operation, three extra messages *Token-Request*, *Token-Grant*, and *Token-Release* have been sent between the *Notifier* and the notifying site. Although the *Token-Release* message can be avoided by piggybacking it in the message that contains propagated operation, and the *Token-Grant* message can be possibly avoided by piggybacking it in the message that contains the operation forwarded from the *Notifier*, there is definitely no way to avoid the *Token-Request* message.

Concurrent propagation So we propose another solution, which allows a site to propagate its notification without first requesting the token, thus effectively eliminating the *Token-Request* message. In this solution, multiple sites could concurrently propagate their notifications to the *Notifier*. As a result, for any O_x in OB_k at site k , not only are operations in IB_k concurrent with O_x but also are those operations on their way towards IB_k .

The proposed solution is called *SCOP* (Symmetric Contextually-serialized Operation Propagation) protocol. As shown in Figure 5, sequential propagation is relaxed to be sequential access to the *Notifier*, that is sequential access to the Incoming Message Queues (*IMQ*) at the *Notifier* for queuing incoming notification messages towards notified sites. The Outgoing Message Queues (*OMQ*) at the *Notifier* are used to queue notification messages propagated from notifying sites. Transit *IMQs/OMQs* (*TIMQ/TOMQ*) are used to illustrate those

notification messages on the way between a notifying site and the *Notifier* or between the *Notifier* and a notified Site. It is those notification messages in *TIMQ/TOMQ* that make concurrency control difficult. One thing that should be clarified is those *IMQs/OMQs* are conceptual data structures for illustration purpose only.

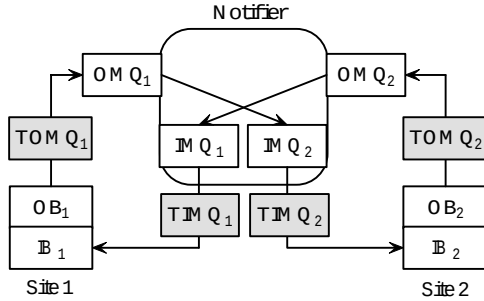


Figure 5: The SCOP propagation protocol

The challenge of this protocol is to ensure each operation would never miss out the chance of being transformed with any concurrent operation no matter where it is. For example, as shown in Figure 5, an operation O_x in OB_1 could be concurrent with another operation O_y , which could be in IB_1 , $TIMQ_1$, IMQ_1 , OMQ_2 , $TOMQ_2$, or OB_2 .

A notification message contains two fields: a list of operations L and a pointer $ILTO$ (Identifier of Last Transformed Operation). $ILTO$ is used to remember the last operation in IB with which operations in L have been transformed. L is achieved with the *AnyONE* algorithm and $ILTO$ is achieved by the *getILTO*(IB , $ILAO$) function defined as follows.

Function 1 *getILTO*(IB , $ILAO$): $ILTO$

```
{ if ( $|IB| == 0$ )
     $ILTO = ILAO$ ;
else
     $ILTO = ID(IB[|IB|-1])$ ;
return  $ILTO$ ;
}
```

As shown in Figure 5, when the notification message: $L = [O''_{N_1}, \dots, O''_{N_k}]$; $ILTO = ID(O_r)$ propagated from Site 1 arrives at the *Notifier*, it will be put in OMQ_1 . When Site 1 gains access to the *Notifier*, the following steps will be executed repeatedly until OMQ_1 is empty: (1) The *removeMessage* procedure is executed to remove messages in IMQ_1 with operations in whose Ls operations in the notification message have been transformed; (2) The *SLOT* algorithm is executed to transform operations in the notification message with the operations in the remaining messages in IMQ_1 ; (3) The *updateILTO* procedure is executed to update the $ILTO$ pointer; (4) The notification message is removed from OMQ_1 , put in the $IMQs$ for all notified sites, and propagated from the *Notifier* to all notified sites, for example, Site 2.

The *removeMessage* procedure and the *updateILTO* procedure are defined as follows.

Procedure 4 *removeMessage*(IMQ , $ILTO$)

```
{ for ( $i = 0$ ;  $i < |IMQ|$ ;  $i++$ )
    if ( $ID(IMQ[i].L[|L|-1]) == ILTO$ ) break;
if ( $i < |IMQ|$ )
    for ( $j = 0$ ;  $j \leq i$ ;  $j++$ ) remove( $IMQ[0]$ );
}
```

Procedure 5 *updateILTO*(IMQ , $ILTO$)

```
{ if ( $|IMQ| > 0$ )
     $ILTO = ID(IMQ[|IMQ|-1].L[|L|-1])$ ;
}
```

When the notification messages arrives at Site 2, the following steps would be done: (1) *removeOperation* procedure is executed to remove operations OB_2 with which operations in the notification message have been transformed; (2) The *SLOT* algorithm is executed to transform operations in the notification message with the remaining **propagated** operations in OB_2 ; (3) Operations in the notification message are appended to IB_2 .

The *removeOperation* procedure is similar to the *removeMessage* procedure, which is defined as follows.

Procedure 6 *removeOperation*(OB , $ILTO$)

```
{ for ( $i = 0$ ;  $i < |OB|$ ;  $i++$ )
    if ( $ID(OB[i]) == ILTO$ ) break;
if ( $i < |OB|$ )
    for ( $j = 0$ ;  $j \leq i$ ;  $j++$ ) remove( $OB[0]$ );
}
```

In summary, the *SCOP* notification propagation protocol is described as follows.

Protocol 1 *SCOP*

- At the notifying site k , *AnyONE* algorithm is executed to get the L field and *getILTO*(IB_k , $ILAO_k$) procedure is executed to get the $ILTO$ field for a notification message. Then the notification message is propagated.
- At the *Notifier*, when the message arrives, it will be put in OMQ_k . When Site k gets access, the following will be done:

```
while ( $|OMQ_k| > 0$ )
{  $Msg = OMQ_k[0]$ ;
  removeMessage( $IMQ_k$ ,  $Msg.ILTO$ );
  for ( $i = 0$ ;  $i < |IMQ_k|$ ;  $i++$ )
      SLOT( $IMQ_k[i].L$ ,  $Msg.L$ );
  updateILTO( $IMQ_k$ ,  $Msg.ILTO$ );
  remove( $OMQ_k[0]$ );
  for (any notified site  $l$ )
  { append( $Msg$ ,  $IMQ_l$ );
    propagate( $Msg$ , Site  $l$ );
  }
}
```
- At the notified site l , when a notification message Msg arrives, the following will be done:

```
removeOperation( $OB_l$ ,  $Msg.ILTO$ );
for ( $i = 0$ ;  $i < |OB_l|$ ;  $i++$ )
{ append( $OB_l[i]$ ,  $tempList$ );
  if ( $ID(OB_l[i]) == ILPO_l$ )
```

```

    break;
}
SLOT(tempList, Msg.L);
for (j = 0; j < |Msg.L|; j++)
    append(Msg.L[j], IBl);

```

Significance of the SLOT algorithm

The *SLOT* transformation control algorithm is much simpler and more efficient than other algorithms such as *GOTO*. There are three other important advantages. Firstly, it is free of state vectors. State vectors are usually needed to capture concurrent relationships among operations, which have been achieved because the notification protocol ensures operations in *OB* and *IB* at the same site are concurrent. Secondly, it is free of *ET* transformation functions. Although theoretically *IT* and *ET* are reversible [16], that is, if $O'_a = IT(O_a, O_b)$, then $O_a = ET(O'_a, O_b)$, this reversibility cannot be always guaranteed in reality due to information loss in *IT* transformations [16].

Finally, it is free of the *TP2* (Transformation Property 2) [15]. *TP2* imposes a condition that the transformation of an operation against a pair of operations is independent of the order the pair of operations are transformed against each other. In practice, it is hard to verify this condition and if it is violated, consistency may not be maintained. The reason why the *SLOT* algorithm is free of *TP2* is that under no circumstance an operation could be transformed against the same pair of operations but in different orders. An operation O in OB_l at site l may be transformed against the set of operations in IB_l , which has a unique order. O may also be transformed against operations in OB_{r_1} at remote site r_1 and operations in OB_{r_2} at remote site r_2 , but operations in OB_{r_1} and OB_{r_2} must be totally different.

AN EXAMPLE

Consider the scenario that such a group editor is used in a brainstorming discussion. The discussion consists of a group of participants and a reviewer who collects ideas, assesses them, and gives comments. Those participants use the notification policy: $ONF = I$, $ONG = A$, $INF = I$, and $ING = A$ to work in real-time mode because they want their ideas to be assessed by the reviewer and receive comments as soon as possible. On the contrary, the reviewer uses another notification policy: $INF = U$, $ING = A$, $ONF = U$, and $ONG = S$ to work in non-real-time mode in order to avoid keeping receiving new messages while s/he is focusing on reviewing current ideas. On the other hand, after reviewing for a period of time, the reviewer will selectively broadcast comments on good ideas.

We take two sites as an example, where the user at *Site 1* is a participant and the user at *Site 2* is a reviewer. The document initially contains “abcd”. The participant performed an operation $O_1 = del[1, 2, bc]$ to delete two characters starting from position 1 (i.e., “bc”). The document then becomes “ad”. Concurrently the reviewer performed an operation $O_2 = del[0, 1, a]$ to delete character “a” and a subsequent oper-

ation $O_3 = ins[2, 1, x]$ to insert a character “x” at position 2 (i.e., between the character “c” and “d”). The document then becomes “bcxd”. According to the notification policy at *Site 1*, operation O_1 will be propagated instantly as generated. Suppose the reviewer happens to selectively propagate O_3 concurrently. As a result, notifications on O_1 and O_3 are propagated to the *Notifier* concurrently. Suppose *Site 1* gains access to the *Notifier* first. As shown in Figure 6, before notifications on O_1 and O_3 , $OB_1 = [O_1]$, $OB_2 = [O_2, O_3]$, and $ILPO_1 = ILAO_1 = ILPO_2 = ILAO_2 = null$.

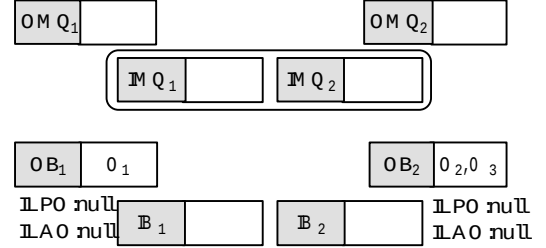


Figure 6: Before AnyONE(O_1) and AnyONE(O_3)

AnyONE(O_1) and AnyONE(O_3)

In *AnyONE*(O_1), because $ONG = A$, there is no need to execute the *LTranspose* procedure. Furthermore, because IB_1 is empty, there is no operation in IB_1 with which O_1 needs to be transformed. As shown in Figure 7, a notification message Msg_1 with $L = [O_1]$ and $ILTO = getILTO(IB_1, ILAO_1) = null$ is then propagated to the *Notifier*. Finally $ILPO_1$ is set to $ID(O_1)$.

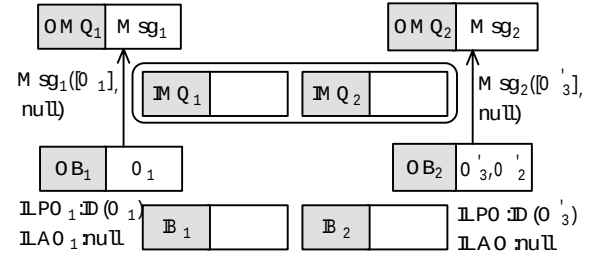


Figure 7: After AnyONE(O_1) and AnyONE(O_3)

In *AnyONE*(O_3), because $ONG = S$, *LTranspose* ($[O_2, O_3]$) is executed to transpose O_3 ahead of O_2 . After then, $OB_2 = [O'_3, O'_2]$ where $O'_3 = ins[3, 1, x]$ and $O'_2 = del[0, 1, a]$. Since IB_2 is also empty, there is no operation in IB_2 with which O'_3 needs to be transformed. As shown in Figure 7, a notification message Msg_2 with $L = [O'_3]$ and $ILTO = getILTO(IB_2, ILAO_2) = null$ is then propagated to the *Notifier*. Finally $ILPO_2$ is set to $ID(O'_3)$.

The propagation of O_1

When Msg_1 arrives at the *Notifier*, it is put into $OM Q_1$. As shown in Figure 8, when *Site 1* gets access, because $IM Q_1$ is empty, there is no need to execute *removeMessage*, *SLOT*, and *updateILTO* procedures. Msg_1 will be removed from

OMQ_1 , put into IMQ_2 , and propagated to Site 2. When Msg_1 arrives at Site 2, because $Msg_1.ILTO = null$, nothing will be removed from OB_2 by the *removeOperation* (OB_2 , $Msg_1.ILTO$) procedure. After the execution of $SLOT([O'_3], Msg_1.L) = SLOT([O'_3], [O_1])$, $OB_2 = [O'_3, O'_2]$ where $O'_3 = ins[1, 1, x]$, and $Mmsg_1.L = [O'_1]$ where $O'_1 = del[1, 2, bc]$, is then appended to IB_2 .

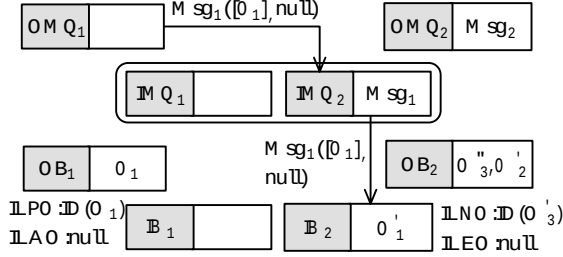


Figure 8: The propagation of O_1

The propagation of O'_3

When $Mmsg_2$ arrives at the Notifier, it is put into OMQ_2 . As shown in Figure 9, when Site 2 gets access, because $Mmsg_2.ILTO = null$, nothing will be removed from IMQ_2 by the *removeMessage* (IMQ_2 , $Mmsg_2.ILTO$) procedure. After the execution of $SLOT(Mmsg_1.L, Mmsg_2.L) = SLOT([O_1], [O'_3])$, $Mmsg_1.L = [O'_1]$ where $O'_1 = del[1, 2, bc]$ and $Mmsg_2.L = [O'_3]$ where $O'_3 = ins[1, 1, x]$. After the execution of *updateILTO* (IMQ_2 , $Mmsg_2.ILTO$), $Mmsg_2.ILTO = ID(O_1)$. Then $Mmsg_2$ is removed from OMQ_2 , put into IMQ_1 , and propagated to Site 1. When it arrives at Site 1, after the execution of *removeOperation* (OB_1 , $Mmsg_2.ILTO$), O_1 is removed from OB_1 , and $ILPO_1 = null$. Because there is no propagated operation in OB_1 , there is no need to execute *SLOT*, and operations in $Mmsg_2.L = [O'_3]$ is appended to IB_1 as is.

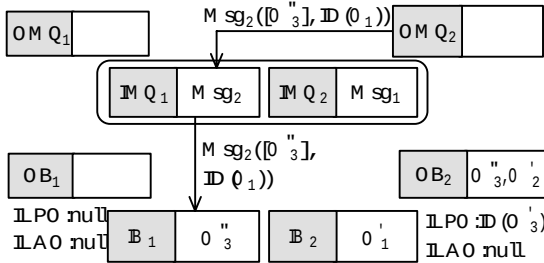


Figure 9: The propagation of O'_3

AnyINE(O'_3) and AnyINE(O'_1)

Because $INF = I$ at Site 1, O'_3 will be executed immediately on arrival. In *AnyINE*(O'_3), because $ING = A$, there is no need to execute the *LTranspose* procedure. Furthermore, because OB_1 is empty, there is no unpropagated operation in OB_1 with which O'_3 needs to be transformed. So $O'_3 = ins[1, 1, x]$ is executed as is on the current document state "ad". After execution, as shown in Figure 10, $ILAO_1 = ID(O'_3)$,

and the document becomes "axd". O_3 generated at Site 2 has been correctly executed at Site 1.

When the reviewer decides to accept notifications from the participant, s/he explicitly issues a command. Because $ING = A$, there is no need to execute the *LTranspose* procedure. As shown in Figure 10, $ILPO = ID(O'_3)$ indicates that O'_2 in OB_2 is an unpropagated operation. So O'_1 in IB_2 needs to be transformed with O'_2 in OB_2 . After the execution of $SLOT([O'_1], [O'_2])$, OB_2 becomes $[O'_3, O'_2]$ where $O'_2 = del[0, 1, a]$. $O'_1 = del[0, 2, bc]$ is executed on the current document state "bcxd". After execution, $ILAO_2 = ID(O'_1)$, and the document becomes "xd". O_1 generated at Site 1 has been correctly executed at Site 2.

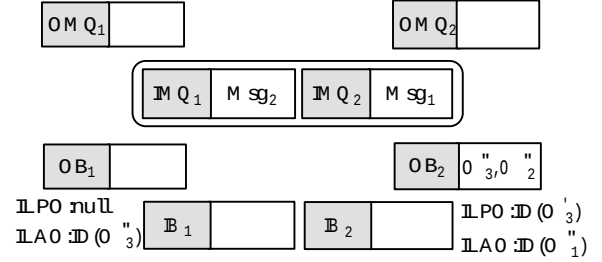


Figure 10: After *AnyINE*(O'_3) and *AnyINE*(O'_1)

A COMPRESSION ALGORITHM

A challenging issue is that notification buffers could grow dramatically in long duration collaboration sessions. We have devised a novel compression algorithm *CLOM* (Compress a Log by Operational Merging) [14] to compress logs for operation-based version control systems [14]. This algorithm is able to minimize the size of a log as well as the number of operations within it by the proposed operational merging technique [14] and the operational transformation technique [16, 15]. The algorithm can be directly applied to the compression of notification buffers.

The operational merging technique is also important for the *LTranspose* procedure to support selective notification. As we know, *ET* transformation may not succeed if the effects of two causal operations are *overlapping* [14]. The operational merging technique can be used to make their effects *disjointed* [14], thus making *ET* transformation successful.

COMPARISON TO RELATED WORK

Some key issues for notification servers have been discussed in [13]. Their work focused on deriving a conceptual framework to guide the design of notification servers while our work focuses on designing algorithms to tackle technical issues involved in notification. Some issues have been raised in [13]. For example, the concept of "pace impedance" has been proposed to allow the notification pace between the notifying site and the notification server to be different from that between the notification server and the notified side. We addressed this issue by separating outgoing and incoming notification policies. Some issues that we have addressed were

not mentioned in [13]. For example, the design of a notification service should consider how to effectively support concurrency control if updates are made on shared artifacts.

NSTP (Notification Service Transfer Protocol) [11] provided a simple and general application-independent notification service for sharing artifacts in synchronous multi-user applications. Its goal was to provide notification service for synchronous applications, therefore it did not address issues related to the support of asynchronous applications. Moreover, because updates are made on shared artifacts, a centralized locking-based serialization protocol was designed for concurrency control. This solution is simple and independent of applications, but it may not be scalable for some applications because the notification server has to maintain all shared artifacts and it could become a bottleneck as the number of shared artifacts grows to some extent.

NESSIE [12] provided an application-independent generic notification infrastructure for asynchronous application sharing systems. Because its goal was to provide task-based group awareness, notifications are consequently made on the basis of tasks instead of individual events. As a result, it is unsuitable to be used for notification service in synchronous applications. Another major difference between [12] and our work is they focused on the issues of using notification to support group awareness while our work focuses on the issues of using notification to support concurrency control.

CONCLUSIONS AND FUTURE WORK

In this paper, we propose a flexible notification framework in which notification policy is separated with notification mechanism. In the policy part, we use two parameters *frequency* and *granularity* to define a spectrum of notification policies. In the mechanism part, separated notification buffers and separated notification executors are used to support various outgoing/incoming notification policies.

We have applied the framework to the design of a notification component for a group editor, which supports a range of notification policies for meeting both real-time and non-real-time collaboration needs. The notification mechanism consists of two algorithms *AnyONE* and *AnyINE*, and a propagation protocol *SCOP*.

In addition, we contribute a new operational transformation control algorithm *SLOT* for concurrency control, which is significantly simpler and more efficient than existing algorithms. Furthermore, it is free of state vectors, free of *ET* transformation functions, and free of the *TP2* transformation condition.

We are now in the process of implementing such an integrated group editor called *NICE* (Notification-flexible Collaborative Editing) system by means of the proposed flexible notification component.

ACKNOWLEDGEMENTS

The work reported in this paper has been partially supported by an ARC (Australian Research Council) Large Grant (000-

00711).

REFERENCES

1. Banavar, G., Chandra, T., Mukherjee, B., and Nagara-jarao, J. An Efficient Multicast Protocol for Content-based Publish-Subscribe Systems, in *Proc. of International Conference on Distributed Computing Systems*, 1999. 262-272.
2. Berliner, B. CVS II: Parallelizing software development, in *Proc. of Winter USENIX'90*. 341-352.
3. Birman, K.P. The process group approach to reliable distributed computing. *Communications of the ACM* 36, 12 (1993), 37-54.
4. Carzaniga, A., Rosenblum, D.S., and Wolf, A.L. Design and Evaluation of a Wide-Area Event Notification Service. *ACM Transaction on Computer Systems* 19, 3 (2001), 332-383.
5. Edwards, W.K., Mynatt, E.D., Petersen, K., Spreitzer, M.J., Terry, D.B., and Theimer, M.M. Designing and Implementing Asynchronous Collaborative Applications with Bayou, in *Proc. of ACM Symposium on User Interface Software and Technology*, 1997. 119-128.
6. Ellis, C.A., and Gibbs, S.J. Concurrency control in groupware systems, in *Proc. of ACM SIGMOD Conference on Management of Data*, 1989. 399-407.
7. ICQ. ICQ is Online Happiness. <http://web.icq.com>.
8. Li, D., Sun, C., Zhou, L., and Muntz, R.R. Operation propagation in real-time group editors. *IEEE Multimedia* 7, 4 (2000), 55-61.
9. Microsoft. Microsoft Windows Netmeeting. <http://www.microsoft.com/windows/netmeeting/>.
10. Myers, E. An $O(ND)$ difference algorithm and its variations. *Algorithmica* 1, 2 (1986), 251-266.
11. Patterson, J.F., Day, M., and Kucan, J. Notification Servers for Synchronous Groupware, in *Proc. of CSCW'96*. 122-129.
12. Prinz, W. NESSIE: An Awareness Environment for Cooperative Settings, in *Proc. of the sixth European conference on Computer Supported Cooperative Work*, 1999. 391-410.
13. Ramduny, D., Dix, A., and Rodden, T. Exploring the design space for notification servers, in *Proc. of CSCW'98*. 227-235.
14. Shen, H., and Sun, C. A Log compression algorithm for operation-based version control systems, in *Proc. of International Computer Software and Application Conference*, 2002. To appear.
15. Sun, C., and Ellis, C.A. Operational transformation in real-time group editors: issues, algorithms, and achievements, in *Proc. of CSCW'98*. 59-68.
16. Sun, C., Jia, X., Zhang, Y., Yang, Y., and Chen, D. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transaction on Computer Human Interaction* 5, 1 (1998), 63-108.